

제2장 : 유한 상태 머신

이 장에서는 이 책에 포함된 간단한 탱크 게임 메커니즘 예제를 통해 Unity3D 게임에 유한 상태 머신 (FSM)을 구현하는 방법을 배우겠습니다 .

우리 게임에서 플레이어는 탱크를 조종합니다. 적 탱크들은 4개의 웨이포인트(waypoints, 경유지)를 따라 씬(scene) 주변을 이동합니다. 플레이어의 탱크가 적 탱크의 시야 범위(vision range)에 들어오면 적들은 추적을 시작하고, 공격할 수 있을 만큼 충분히 가까워지면 플레이어의 탱크를 향해 사격을 시작합니다.

적 탱크의 AI를 제어하기 위해 우리는 FSM을 사용합니다. 먼저, 간단한 switch 문을 사용하여 탱크 AI 상태를 구현할 것입니다. 그런 다음, 캐릭터의 FSM을 설계할 때 더 큰 유연성을 제공하는 좀 더 복잡하고 잘 구조화된(engineered) FSM 프레임워크(framework)를 사용할 것입니다.

이 장에서 다룰 주제는 다음과 같습니다.

- 플레이어 탱크 구현하기
- 총알(Bullet) 클래스 구현하기
- 웨이포인트 설정하기
- 추상 FSM 클래스 생성하기
- 적 탱크 AI에 간단한 FSM 사용하기
- FSM 프레임워크 사용하기

기술 요구사항

이 장을 위해서는 Unity3D 2022만 있으면 됩니다. 이 장에서 설명하는 예제 프로젝트는 책 저장소의 **Chapter 2** 폴더에서 찾을 수 있습니다.<https://github.com/PacktPublishing/Unity-Artificial-Intelligence-Programming-Fifth-Edition/tree/main/Chapter02> .

플레이어의 탱크 구현

플레이어 탱크를 위한 스크립트를 작성하기 전에, PlayerTank 게임 오브젝트(game object)를 어떻게 설정했는지 살펴보겠습니다. 우리의 Tank 오브젝트는 리지드바디(Rigidbody)와 박스 콜라이더(Box Collider) 컴포넌트를 가진 단순한 메시(mesh)입니다.

Tank 오브젝트는 Tank와 Turret(포탑)이라는 두 개의 분리된 메시로 구성되며, Turret은 Tank의 자식(child) 오브젝트입니다. 이러한 구조는 마우스 움직임을 사용하여 Turret 오브젝트를 독립적으로 회전시킬 수 있게 해주며, 동시에 Tank 본체가 어디로 가든 자동으로 따라가게 해줍니다. 그런 다음, SpawnPoint 트랜스폼(transform)을 위한 빈 게임 오브젝트를 생성합니다. 우리는 이를 총알을 발사할 때의 기준 위치 포인트로 사용합니다. 마지막으로, Tank 오브젝트에 Player 태그(tag)를 할당해야 합니다. 이제 컨트롤러 클래스를 살펴보겠습니다.

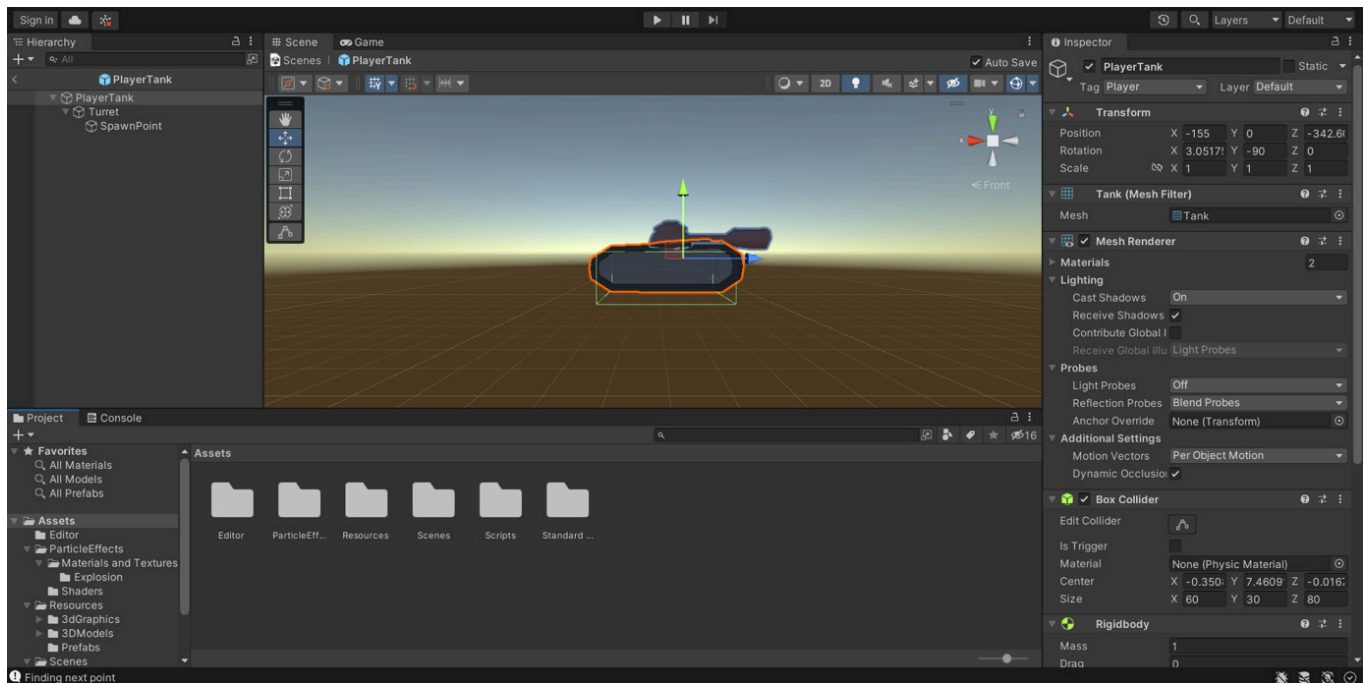


그림 2.1 – 우리의 탱크 개체

PlayerTankController 클래스 는 플레이어의 탱크를 제어합니다. *W*, *A*, *S*, *D* 키를 사용하여 탱크를 이동하고 조향(steer)하며, 마우스 왼쪽 버튼을 사용하여 Turret 오브젝트에서 조준하고 사격합니다.

정보

이 책에서는 QWERTY 키보드와 왼쪽 마우스 버튼이 기본 버튼으로 설정된 두 버튼 마우스를 사용한다고 가정합니다 . 만약 다른 키보드를 사용하고 있다면, *QWERTY* 키보드를 사용하는 것처럼 생각하거나 , *QWERTY* 키보드를 사용하는 것과 유사한 방식으로 따라 하면 됩니다.키보드 레이아웃에 맞게 코드를 수정하세요. 아주 간단합니다!

탱크 오브젝트 초기화하기 (Initializing the Tank object)

PlayerTankController.cs 파일에 Start 함수와 Update 함수를 설정하여 PlayerTankController 클래스 생성을 시작해 봅시다:

```
using UnityEngine;

using System.Collections;

public class PlayerTankController : MonoBehaviour {

    public GameObject Bullet;

    public GameObject Turret;

    public GameObject bulletSpawnPoint;
```

```
public float rotSpeed = 150.0f;

    public float turretRotSpeed = 10.0f;

public float maxForwardSpeed = 300.0f;

public float maxBackwardSpeed = -300.0f;

public float shootRate = 0.5f;

private float curSpeed, targetSpeed;

protected float elapsedTime;

void Start() {

}

void Update() {

    UpdateWeapon();

    UpdateControl();

}
```

계층 구조(hierarchy)를 보면 PlayerTank 게임 오브젝트에 Turret이라는 자식이 하나 있고, 이 Turret 오브젝트의 첫 번째 자식 이름이 SpawnPoint인 것을 볼 수 있습니다. 컨트롤러를 설정하려면 Turret과 SpawnPoint를 드래그 앤 드롭하여 인스펙터(Inspector)의 해당 필드에 연결해야 합니다.

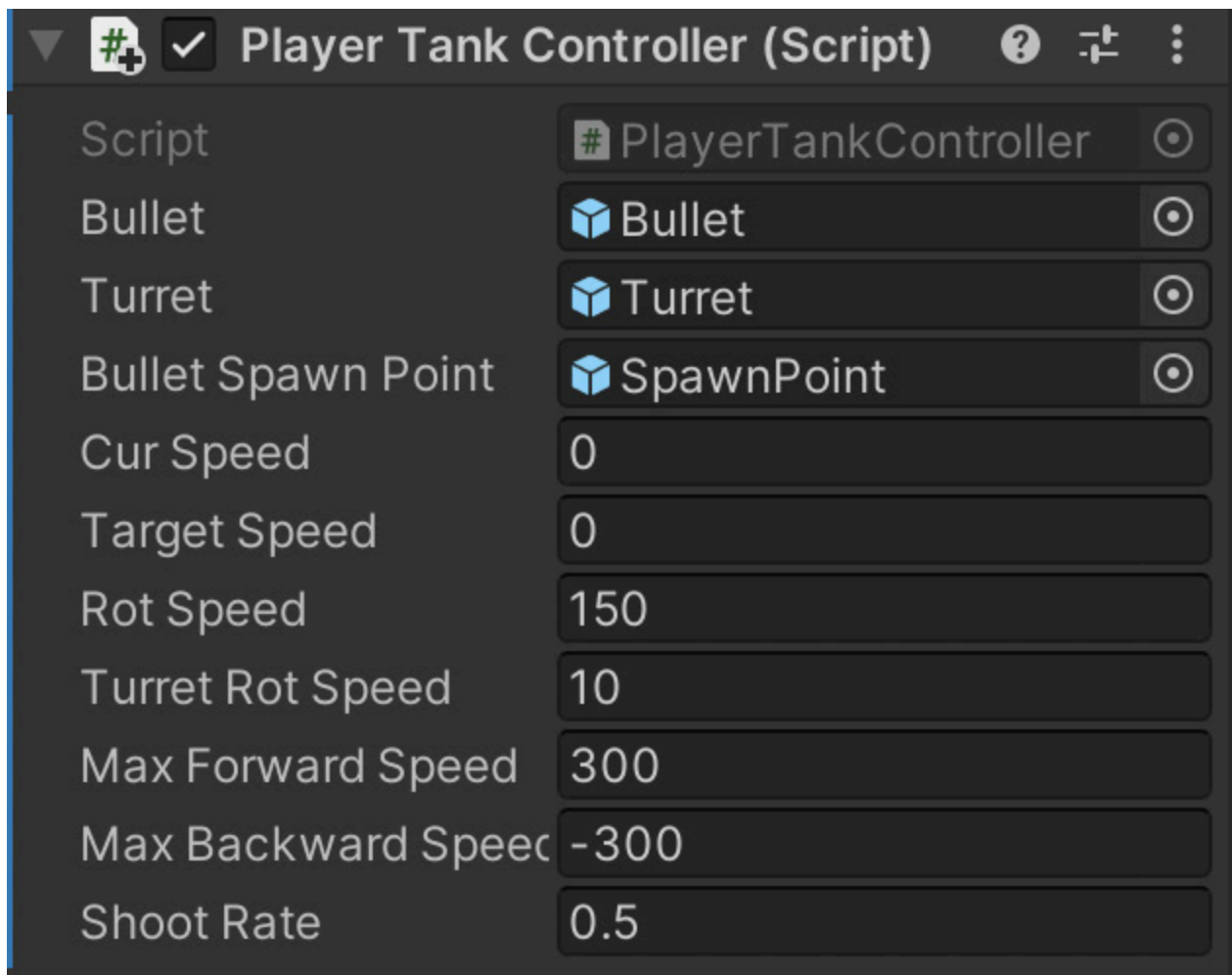


그림 2.2 – 인스펙터의 Player Tank Controller 컴포넌트

나중에 Bullet 오브젝트를 생성한 후, 인스펙터를 사용하여 이를 Bullet 변수에 할당할 수 있습니다. 마지막으로 Update 함수는 UpdateControl 및 UpdateWeapon 함수를 호출합니다. 이 함수들의 내용은 다음 섹션에서 다룰 것입니다.

총알 발사하기 (Shooting the bullet)

총알을 발사하는 메커니즘은 간단합니다. 플레이어가 마우스 왼쪽 버튼을 클릭할 때마다, 마지막 발사 이후 경과된 총시간이 무기의 발사 속도(fire rate)보다 큰지 확인합니다. 만약 그렇다면, bulletSpawnPoint 트랜스폼의 위치에 새로운 Bullet 오브젝트를 생성(Instantiate)합니다. 이 확인 과정은 플레이어가 연속해서 총알을 난사하는 것을 방지합니다.

이를 위해 PlayerTankController.cs 파일에 다음 함수를 추가합니다:

```
void UpdateWeapon() {

    elapsedTime += Time.deltaTime;

    if (Input.GetMouseButtonDown(0)) {
```

```

        if (elapsedTime >= shootRate) {

            //시간을 재설정합니다

            elapsedTime = 0.0f;

            // 총알 객체를 인스턴스화합니다

            Instantiate(Bullet,

                bulletSpawnPoint.transform.position,

                bulletSpawnPoint.transform.rotation);

        }

    }

}

```

이제 이 컨트롤러 스크립트를 PlayerTank 오브젝트에 연결(attach)할 수 있습니다. 게임을 실행하면 탱크에서 사격할 수 있을 것입니다. 이제 탱크의 이동 컨트롤을 구현할 차례입니다.

탱크 제어하기 (Controlling the tank)

플레이어는 마우스를 사용하여 Turret 오브젝트를 회전시킬 수 있습니다. 이 부분은 레이캐스팅(raycasting)과 3D 회전을 포함하기 때문에 약간 까다로울 수 있습니다. 카메라가 전장(battlefield)을 내려다보고 있다고 가정합니다. PlayerTankController.cs 파일에 UpdateControl 함수를 추가해 봅시다:

```

void UpdateControl() {

    // 마우스로 조준하기

    // 변환과 교차하는 평면을 생성합니다.

    // 위쪽 법선을 기준으로 위치를 지정합니다.

    Plane playerPlane = new Plane(Vector3.up,

        transform.position + new Vector3(0, 0, 0));

    // 커서 위치에서 광선을 생성합니다

    Ray RayCast =

```

```

        Camera.main.ScreenPointToRay(Input.mousePosition);

// 커서 광선이 교차하는 지점을 확인합니다.

// 비행기.

float HitDist = 0;

// 광선이 평면에 평행하면 Raycast는

// false를 반환합니다.

if (playerPlane.Raycast(RayCast, out HitDist)) {

    // 광선이 만나는 지점을 가져옵니다.

    // 계산된 거리.

    Vector3 RayHitPoint = RayCast.GetPoint(HitDist);

    Quaternion targetRotation =

        Quaternion.LookRotation(RayHitPoint -

                                transform.position);

    Turret.transform.rotation = Quaternion.Slerp(

        Turret.transform.rotation, targetRotation,

        Time.deltaTime * turretRotSpeed);

}

}

```

전장에서 마우스 위치(mousePosition) 좌표를 찾아 회전 방향을 결정하기 위해 레이캐스팅을 사용합니다.

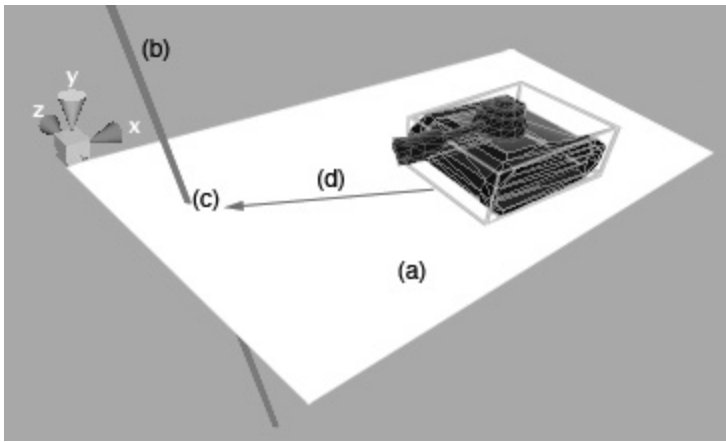


그림 2.3 – 마우스로 조준하기 위한 레이캐스트

정보

레이캐스팅은 유니티 물리 엔진(physics engine)에서 기본으로 제공하는 도구입니다. 이를 통해 씬(scene) 내의 가상의 선(레이, ray)과 콜라이더(collider) 사이의 교차점(intersection point)을 찾을 수 있습니다. 이를 레이저 포인터라고 상상해 보세요. 특정 방향으로 레이저를 쏘고 그것이 부딪히는 지점을 볼 수 있습니다. 그러나 이것은 상대적으로 비용이 많이 드는(expensive) 연산입니다. 일반적으로 프레임당 100~200개의 레이캐스트는 무리 없이 처리할 수 있지만, 레이의 길이나 씬에 있는 콜라이더의 수와 유형에 따라 성능이 크게 영향을 받습니다. 따라서 팁을 드리자면, 메시 콜라이더(mesh colliders)에 너무 많은 레이캐스트를 사용하지 않도록 하고, 레이어 마스크(layer masks)를 사용하여 불필요한 콜라이더를 필터링하십시오.

작동 방식은 다음과 같습니다.

1. 위쪽 방향의 법선 벡터를 사용하여 플레이어 탱크와 교차하는 평면(plane)을 설정합니다.
2. 스크린 공간(screen space)에서 마우스 위치를 향해 레이를 쏩니다(위 다이어그램에서는 탱크를 내려다보고 있다고 가정합니다).
3. 레이가 평면과 교차하는 지점을 찾습니다.
4. 마지막으로 현재 위치에서 해당 교차점까지의 회전을 구합니다.

다음으로, 키 입력을 확인하고 그에 따라 탱크를 이동시키거나 회전시킵니다. **UpdateControl** 함수의 끝에 다음 코드를 추가합니다 .

```
if (Input.GetKey(KeyCode.W)) {
    targetSpeed = maxForwardSpeed;
} else if (Input.GetKey(KeyCode.S)) {
    targetSpeed = maxBackwardSpeed;
} else {
```

```

        targetSpeed = 0;
    }

    if (Input.GetKey(KeyCode.A)) {

        transform.Rotate(0, -rotSpeed * Time.deltaTime, 0.0f);

    } else if (Input.GetKey(KeyCode.D)) {

        transform.Rotate(0, rotSpeed * Time.deltaTime, 0.0f);

    }

    //현재 속도 결정 (Determine current speed)

    curSpeed = Mathf.Lerp(curSpeed, targetSpeed, 7.0f *

        Time.deltaTime);

    transform.Translate(Vector3.forward * Time.deltaTime *

        curSpeed);

```

위의 코드는 고전적인 WASD 제어 방식을 나타냅니다. 탱크는 A와 D 키로 회전하고, W와 S 키로 앞뒤로 이동합니다.

정보

유니티 숙련도에 따라 Lerp 및 Time.deltaTime 곱셈이 무엇인지 궁금할 수 있습니다. 잠시 짚고 넘어갈 가치가 있습니다. 먼저 Lerp는 선형 보간(Linear Interpolation)의 약자이며 두 값 사이를 부드럽게 전환하는 방법입니다. 이전 코드에서는 여러 프레임에 걸쳐 속도 변화를 부드럽게 분산시켜 탱크의 움직임이 순식간에 가속 및 감속되는 것처럼 보이지 않게 하기 위해 Lerp 함수를 사용했습니다. 7.0f 값은 단순한 스무딩 팩터(smoothing factor)이며, 가장 마음에 드는 값을 찾기 위해 숫자를 바꿔가며 테스트해 볼 수 있습니다(값이 클수록 탱크의 가속도가 커집니다).

그런 다음 모든 것에 Time.deltaTime을 곱합니다. 이 값은 현재 프레임과 이전 프레임 사이의 시간(초)을 나타내며, 속도를 프레임 속도(frame rate)에 독립적으로 만들기 위해 사용합니다. 자세한 정보는 <https://learn.unity.com/tutorial/delta-time> 을 참조하세요.

다음으로, 플레이어와 적 탱크가 발사하는 발사체(projectiles)를 구현할 차례입니다.

Bullet 클래스 구현하기

다음으로, 레이저와 같은 재질(material)과 Shader 필드에 Particles/Additive-Layer 속성을 사용하여 두 개의 직교하는 평면과 박스 콜라이더로 Bullet 프리팹(prefab)을 설정합니다.

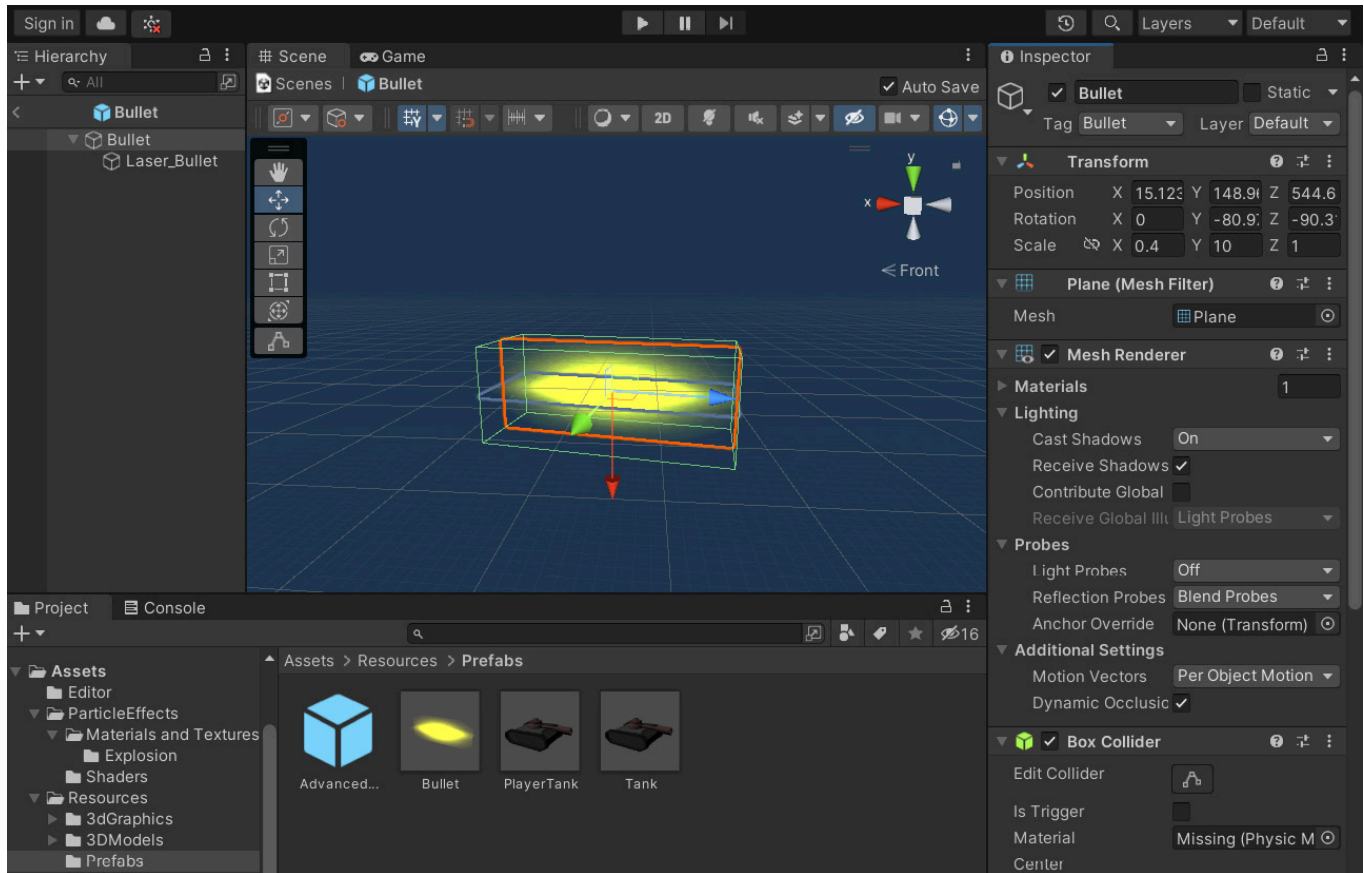


그림 2.4 – 우리의 Bullet 프리팹

Bullet.cs 파일의 코드는 다음과 같습니다:

```
using UnityEngine;

using System.Collections;

public class Bullet : MonoBehaviour {

    //폭발 효과

    [SerializeField] // 인스펙터에서 비공개로 노출하기 위해 사용됨

    // fields!

    private GameObject Explosion;

    [SerializeField]

    private float Speed = 600.0f;
```

```

[SerializeField]

private float LifeTime = 3.0f;

public int damage = 50;

void Start() {

    Destroy(gameObject, LifeTime);

}

void Update() {

    transform.position +=

    transform.forward * Speed * Time.deltaTime;

}

void OnCollisionEnter(Collision collision) {

    ContactPoint contact = collision.contacts[0];

    Instantiate(Explosion, contact.point,

                Quaternion.identity);

    Destroy(gameObject);

}

}

```

Bullet 클래스에는 damage, Speed, Lifetime이라는 세 가지 속성이 있습니다. 마지막 속성은 일정 시간이 지난 후 총알이 자동으로 파괴되도록 하기 위함입니다. 인스펙터에 private 필드를 표시하기 위해 [SerializeField]를 사용했다는 점에 유의하세요. 실제로 유니티는 기본적으로 public 필드만 표시합니다. 다른 클래스에서 접근해야 하는 필드만 public으로 설정하는 것이 좋은 관행(good practice)입니다.

보시다시피, 총알의 Explosion 속성은 ParticleExplosion 프리팹에 연결되어 있는데, 이에 대해서는 자세히 논의하지 않겠습니다. 이 프리팹은 ParticleEffects 폴더에 있으므로 Shader 필드에 드롭합니다. 그런 다음, OnCollisionEnter 메서드에 설명된 대로 총알이 무언가에 부딪히면 이 파티클 효과를 재생합니다. ParticleExplosion 프리팹은 AutoDestruct 스크립트를 사용하여 짧은 시간 후에 Explosion 오브젝트를 자동으로 파괴합니다:

```
using UnityEngine;

public class AutoDestruct : MonoBehaviour {

    [SerializeField]

    private float DestructTime = 2.0f;

    void Start() {

        Destroy(gameObject, DestructTime);

    }

}
```

AutoDestruct 스크립트는 작지만 매우 편리합니다. 단지 지정된 초(seconds)가 지난 후 연결된 오브젝트를 파괴할 뿐입니다. 많은 유니티 게임들이 다양한 상황에서 이와 비슷한 스크립트를 빈번하게 사용합니다.

이제 사격과 이동이 가능한 탱크가 생겼으니, 적 탱크를 위한 간단한 정찰 경로(patrolling path)를 설정할 수 있습니다.

경유지 설정

기본적으로 적 탱크는 게임 아레나를 정찰합니다. 이 행동을 구현하려면 먼저 정찰 경로를 지정해야 합니다. 경로 추종(Path following)에 대해서는 6장, '경로 추종 및 스티어링 동작'에서 자세히 살펴볼 것입니다. 지금은 간단한 웨이포인트 경로로 제한하겠습니다.

이를 구현하기 위해 4개의 큐브(Cube) 게임 오브젝트를 임의의 장소에 배치합니다. 이들은 우리 씬 내부의 웨이포인트를 나타내므로 각각의 이름을 WanderPoint로 지정합니다:

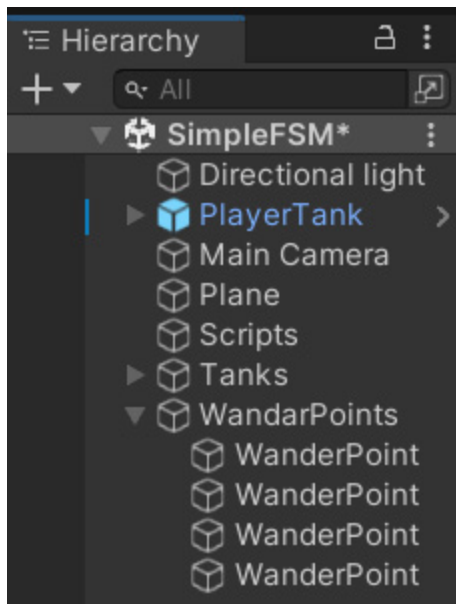


그림 2.5 – 원더포인트

우리의 WanderPoint 오브젝트의 모습은 다음과 같습니다:

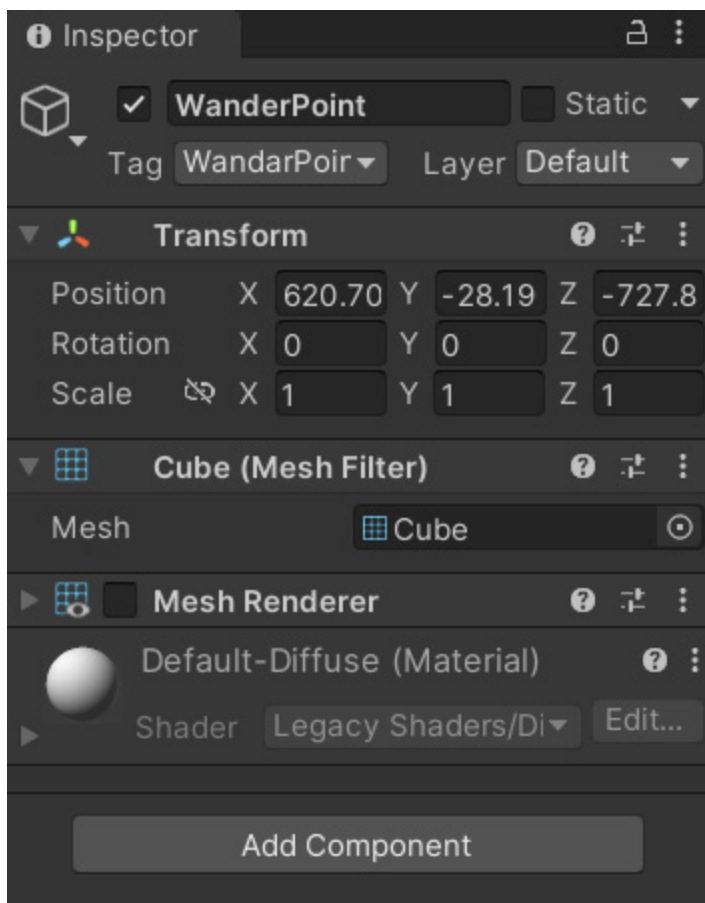


그림 2.6 – WanderPoint 속성

이 포인트들에 WanderPoint라는 태그를 지정해야 한다는 점에 유의하세요. 나중에 탱크 AI에서 웨이포인트를 찾으려고 할 때 이 태그를 사용할 것입니다. 속성에서 볼 수 있듯이 웨이포인트는 Mesh Renderer 체크박스가 비활성화된 큐브 게임 오브젝트일 뿐입니다.

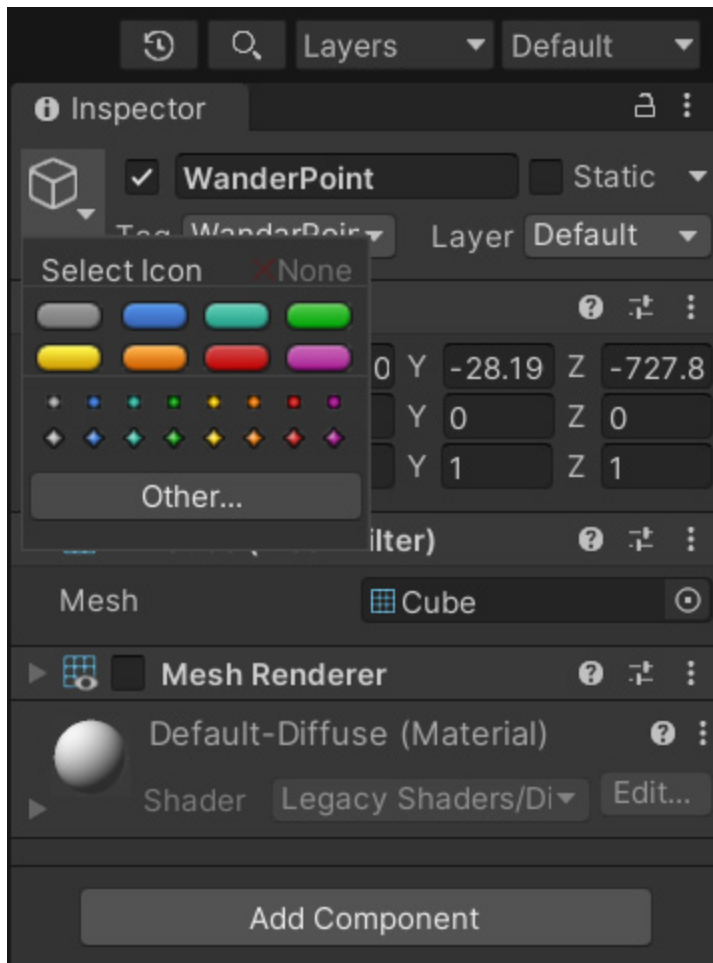


그림 2.7 – 기즈모 선택 패널

이 포인트들을 게임 내에서가 아닌 에디터(editor)에서만 표시하기 위해 기즈모(gizmo) 아이콘이 있는 빈 오브젝트를 사용합니다. 웨이포인트에서 필요한 것은 오직 그 위치와 트랜스폼 데이터뿐이기 때문입니다. 이를 설정하려면 그림 2.7에 표시된 것처럼 인스펙터의 오브젝트 아이콘 근처에 있는 작은 삼각형을 클릭하세요.

이제 FSM의 힘을 빌려 적 탱크에 생명력을 불어넣을 준비가 되었습니다.

추상 FSM 클래스 생성(Creating the abstract FSM class)

다음으로, 적 탱크 AI 클래스의 메서드를 정의하기 위한 범용 추상 클래스(abstract class)를 구현합니다. 이 추상 클래스는 우리 AI의 뼈대(skeleton)가 될 것이며, 적 탱크가 수행해야 할 작업에 대한 높은 수준의 관점(high-level view)을 제공합니다.

FSM.cs 파일에서 이 클래스의 코드를 볼 수 있습니다:

```
using UnityEngine;

using System.Collections;

public class FSM : MonoBehaviour {
```

```

protected virtual void Initialize() { }

protected virtual void FSMUpdate() { }

protected virtual void FSMFixedUpdate() { }

// Use this for initialization

void Start () {

    Initialize();

}

// Update is called once per frame

void Update () {

    FSMUpdate();

}

void FixedUpdate() {

    FSMFixedUpdate();

}

}

```

적 탱크는 플레이어 탱크의 위치, 다음 목적지 지점, 정찰 중에 선택할 웨이포인트 목록만 알면 됩니다. 플레이어 탱크가 사거리 내에 들어오면 포탑 오브젝트를 회전시키고 발사 속도에 맞춰 총알 스폰 포인트에서 사격을 시작합니다.

이전에 설명했듯이, 우리는 이 클래스를 두 가지 방식으로 확장할 것입니다. 하나는 간단한 if-then-else 기반의 FSM(SimpleFSM 클래스)을 사용하는 것이고, 다른 하나는 더 구조화되었지만 더 유연한 FSM(AdvancedFSM)을 사용하는 것입니다. 이 두 가지 FSM 구현체는 FSM 추상 클래스를 상속(inherit)하며, Initialize, FSMUpdate, FSMFixedUpdate라는 세 가지 추상 메서드를 오버라이드하여 구현하게 됩니다.

다음 섹션에서 이 세 가지 메서드를 구현하는 두 가지 다른 방법을 살펴볼 것입니다. 지금은 기본적인 구현부터 시작하겠습니다.

적 탱크 AI에 간단한 FSM 사용하기 (Using a simple FSM for the enemy tank AI)

AI 탱크의 실제 코드를 살펴보겠습니다. 먼저 FSM 추상 클래스를 상속하는 SimpleFSM이라는 새로운 클래스를 만듭니다.

SimpleFSM.cs 파일에서 소스 코드를 찾을 수 있습니다:

```
using UnityEngine;

using System.Collections;

public class SimpleFSM : FSM {

    public enum FSMState {

        None, Patrol, Chase, Attack, Dead,

    }

    //NPC가 현재 도달하고 있는 상태

    public FSMState curState = FSMState.Patrol;

    //탱크의 속도

    private float curSpeed = 150.0f;

    //탱크 회전 속도

    private float curRotSpeed = 2.0f;

    //총알

    public GameObject Bullet;

    //NPC가 파괴되었는지 여부

    private bool bDead = false;

    private int health = 100;

    // 더 이상 사용되지 않는 내장 rigidbody를 덮어씁니다.

    // variable.
```

```
new private Rigidbody rigidbody;

//Player Transform

protected Transform playerTransform;

//NPC 탱크의 다음 목적지 위치

protected Vector3 destPos;

//순찰 지점 목록

protected GameObject[] pointList;

//총알 발사 속도

protected float shootRate = 3.0f;

protected float elapsedTime = 0.0f;

public float maxFireAimError = 0.001f;

// Status Radius

public float patrollingRadius = 100.0f;

public float attackRadius = 200.0f;

public float playerNearRadius = 300.0f;

//Tank Turret

public Transform turret;

public Transform bulletSpawnPoint;
```

여기서 몇 가지 변수를 선언합니다. 우리의 탱크 AI에는 정찰(Patrol), 추적(Chase), 공격(Attack), 사망(Dead)이라는 4가지 상태가 있습니다. 우리는 1장, 'AI 소개'에서 예제로 설명했던 FSM을 구현하고 있습니다:

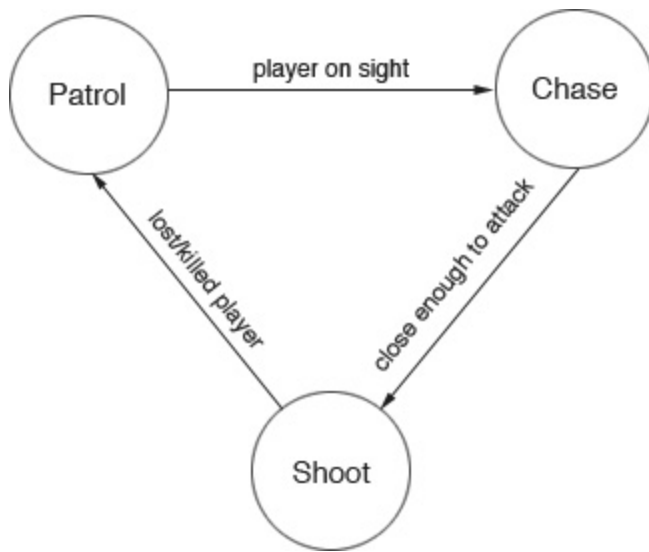


그림 2.8 – 적 탱크 AI의 FSM

Initialize 메서드에서는 기본적으로 AI 탱크의 속성을 설정합니다. 그런 다음 웨이포인트의 위치를 로컬 변수에 저장합니다. FindGameObjectsWithTag 메서드를 사용하여 WanderPoint 태그가 있는 오브젝트를 찾아 씬에서 해당 웨이포인트를 가져옵니다:

```
//NPC 탱크를 위한 유한 상태 기계 초기화

override void Initialize () {

    // 포인트 목록 가져오기 (Get the list of points)

    pointList =

        GameObject.FindGameObjectsWithTag("WanderPoint");

    // 먼저 임의의 목적지 포인트 설정 (Set Random destination point first)

    FindNextPoint();

    // 타겟 적(플레이어) 가져오기 (Get the target enemy(Player))

    GameObject objPlayer =

        GameObject.FindGameObjectWithTag("Player");

    // Get the rigidbody

    rigidbody = GetComponent<Rigidbody>();

    playerTransform = objPlayer.transform;
```

```
if (!playerTransform) {  
  
    print("Player doesn't exist. Please add one with  
        Tag named 'Player'");  
  
}  
  
}
```

매 프레임마다 호출되는 Update 메서드는 다음과 같습니다:

```
protected override void FSMUpdate() {  
  
    switch (curState) {  
  
        case FSMState.Patrol:  
  
            UpdatePatrolState();  
  
            break;  
  
        case FSMState.Chase:  
  
            UpdateChaseState();  
  
            break;  
  
        case FSMState.Attack:  
  
            UpdateAttackState();  
  
            break;  
  
        case FSMState.Dead:  
  
            UpdateDeadState();  
  
            break;  
  
    }  
  
    // Update the time  
  
    elapsedTime += Time.deltaTime;
```

```
// Go to dead state is no health left

if (health <= 0) {

    curState = FSMState.Dead;

}

}
```

현재 상태를 확인한 다음 적절한 상태 메서드를 호출합니다. health 오브젝트의 값이 0 이하가 되면 탱크를 Dead 상태로 설정합니다.

프라이빗 변수 디버깅

인스펙터의 public 변수는 다른 값을 빠르게 실험할 수 있기 때문에 유용할 뿐만 아니라, 디버깅할 때 그 값을 한눈에 빠르게 확인할 수 있기 때문에 유용합니다. 이런 이유로 컴포넌트의 사용자가 변경해서는 안 되는 변수조차 public으로 만들거나(또는 인스펙터에 노출시키고 싶은) 유혹을 느낄 수 있습니다. 걱정하지 마세요. 해결책이 있습니다. 인스펙터를 디버그(Debug) 모드로 표시할 수 있습니다. 디버그 모드에서는 인스펙터가 private 필드도 표시합니다. 디버그 모드를 활성화하려면 우측 상단의 점 3개를 클릭한 다음 **Debug**를 클릭하세요.

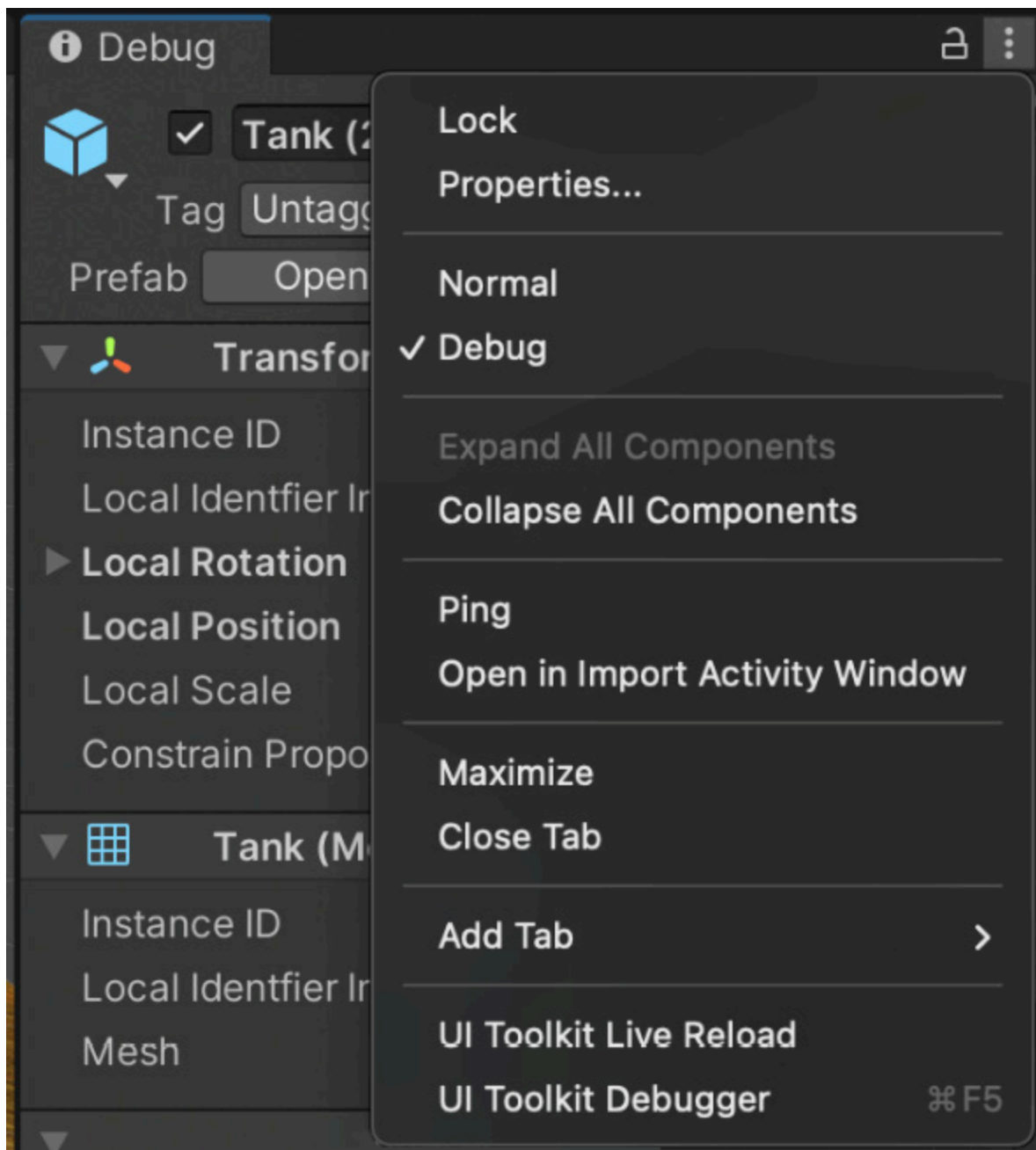


그림 2.9 – 디버그 모드에서 Unity의 인스펙터

이제 각 상태를 하나씩 구현하는 방법을 알아보겠습니다.

Patrol (정찰) 상태

Patrol 상태는 탱크가 웨이포인트 사이를 이동하며 플레이어를 찾는 상태입니다. Patrol 상태의 코드는 다음과 같습니다:

```
protected void UpdatePatrolState() {

    if (Vector3.Distance(transform.position, destPos) <=
        patrollingRadius) {
```

```

        print("목적지 지점에 도달함\n
              다음 지점 계산 중");

        FindNextPoint();

    } else if (Vector3.Distance(transform.position,
                                playerTransform.position) <= playerNearRadius) {

        print("Chase 위치로 전환");

        curState = FSMState.Chase;

    }

    // Rotate to the target point

    Quaternion targetRotation = Quaternion.LookRotation(
        destPos - transform.position);

    transform.rotation = Quaternion.Slerp(
        transform.rotation, targetRotation,
        Time.deltaTime * curRotSpeed);

    // Go Forward

    transform.Translate(Vector3.forward * Time.deltaTime *
                        curSpeed);

}

protected void FindNextPoint() {

    print("다음 지점 찾는 중");

    int rndIndex = Random.Range(0, pointList.Length);

    float rndRadius = 10.0f;

    Vector3 rndPosition = Vector3.zero;

```

```

destPos = pointList[rndIndex].transform.position +

    rndPosition;

// Check Range to decide the random point as the same

// as before

if (IsInCurrentRange(destPos)) {

    rndPosition = new Vector3(Random.Range(-rndRadius,

        rndRadius), 0.0f, Random.Range(-rndRadius,

        rndRadius));

    destPos = pointList[rndIndex].transform.position +

        rndPosition;

}

}

protected bool IsInCurrentRange(Vector3 pos) {

    float xPos = Mathf.Abs(pos.x - transform.position.x);

    float zPos = Mathf.Abs(pos.z - transform.position.z);

    if (xPos <= 50 && zPos <= 50) return true;

    return false;

}

```

탱크가 Patrol 상태에 있는 동안, 목적지 지점에 도달했는지(즉, 탱크가 목적지 웨이포인트에서 100 유닛 이하 거리에 있는지) 확인합니다. 도달했다면 FindNextPoint 메서드를 사용하여 도달할 다음 지점을 찾습니다. 이 메서드는 이전에 정의한 웨이포인트 중에서 임의의 지점을 단순히 선택합니다.

반면에 탱크가 아직 목적지에 도달하지 않았다면, 플레이어 탱크와의 거리를 확인합니다. 플레이어의 탱크가 사거리 내에 있다면(이 예제에서는 300 유닛으로 설정함), AI는 Chase 상태로 전환(switch)합니다. 마지막으로 UpdatePatrolState 함수의 나머지 코드를 사용하여 탱크를 회전시키고 다음 웨이포인트를 향해 이동시킵니다.

Chase (추적) 상태

Chase 상태에서 탱크는 능동적으로 플레이어 탱크에 접근하려고 시도합니다. 간단히 말해, 목적지 지점이 플레이어 탱크 자체가 됩니다. Chase 상태의 구현 코드는 다음과 같습니다:

```
protected void UpdateChaseState() {  
  
    // 타겟 위치를 플레이어 위치로 설정  
  
    destPos = playerTransform.position;  
  
    // 플레이어 탱크와의 거리 확인.  
  
    // 거리가 가까워지면 attack 상태로 전환  
  
    float dist = Vector3.Distance(transform.position,  
  
        playerTransform.position);  
  
    if (dist <= attackRadius) {  
  
        curState = FSMState.Attack;  
  
    } else if (dist >= playerNearRadius {  
  
        curState = FSMState.Patrol;  
  
    }  
  
    transform.Translate(Vector3.forward * Time.deltaTime *  
  
        curSpeed);  
  
}
```

이 상태에서는 먼저 목적지 지점을 플레이어로 설정합니다. 그런 다음 플레이어와 탱크의 거리를 계속 확인합니다. 플레이어가 충분히 가까우면 AI는 Attack 상태로 전환합니다. 반면, 플레이어 탱크가 무사히 도망쳐 너무 멀어지면 AI는 다시 Patrol 상태로 돌아갑니다.

Attack(공격) 상태

Attack 상태는 바로 여러분이 예상하는 바와 같습니다. 적 탱크가 조준하고 플레이어에게 사격합니다. 다음 코드 블록은 Attack 상태를 위한 구현 코드입니다:

```

protected void UpdateAttackState() {

    destPos = playerTransform.position;

    Vector3 frontVector = Vector3.forward;

    float dist = Vector3.Distance(transform.position,

        playerTransform.position);

    if (dist >= attackRadius && dist < playerNearRadius {

        Quaternion targetRotation =

            Quaternion.FromToRotation(destPos -

                transform.position);

        transform.rotation = Quaternion.Slerp(

            transform.rotation, targetRotation,

            Time.deltaTime * curRotSpeed);

        transform.Translate(Vector3.forward *

            Time.deltaTime * curSpeed);

        curState = FSMState.Attack;

    } else if (dist >= playerNearRadius) {

        curState = FSMState.Patrol;

    }

    // 타겟 지점으로 포탑 회전
    // 회전은 오직 탱크의 수직 축을 중심으로만 이루어짐.

    Quaternion targetRotation = Quaternion.FromToRotation(

        frontVector, destPos - transform.position);

    turret.rotation = Quaternion.Slerp(turret.rotation,

        turretRotation, Time.deltaTime * curRotSpeed);

```



```

// 총알 사격

if (Mathf.Abs(Quaternion.Dot(turretRotation,

    turret.rotation)) > 1.0f - maxFireAimError) {

    ShootBullet();

}

}

private void ShootBullet() {

    if (elapsedTime >= shootRate) {

        Instantiate(Bullet, bulletSpawnPoint.position,

            bulletSpawnPoint.rotation);

        elapsedTime = 0.0f;

    }

}

```

첫 번째 줄에서는 여전히 목적지 지점을 플레이어의 위치로 설정합니다. 어쨌든 공격 중에도 플레이어와 가까운 거리를 유지해야 하기 때문입니다. 그런 다음 플레이어 탱크가 충분히 가까우면 AI 탱크는 포탑 오브젝트를 플레이어 탱크 방향으로 회전시킨 다음 사격을 시작합니다. 마지막으로 플레이어의 탱크가 사거리를 벗어나면 탱크는 Patrol 상태로 돌아갑니다.

Dead (사망) 상태

Dead 상태는 최종 상태입니다. 탱크가 Dead 상태가 되면 폭발하고 생성 해제(uninstantiated, 즉 파괴) 됩니다. 다음은 Dead 상태를 위한 코드입니다:

```

protected void UpdateDeadState() {

    // Show the dead animation with some physics effects

    if (!bDead) {

        bDead = true;
    }
}

```

```

        Explode();

    }

}

```

보시다시피 코드는 직관적입니다. 탱크가 Dead 상태에 도달하면 폭발하게 만듭니다:

```

protected void Explode() {

    float rndX = Random.Range(10.0f, 30.0f);

    float rndZ = Random.Range(10.0f, 30.0f);

    for (int i = 0; i < 3; i++) {

        rigidbody.AddExplosionForce(10000.0f,

            transform.position - new Vector3(rndX,

                10.0f, rndZ), 40.0f, 10.0f);

        rigidbody.velocity = transform.TransformDirection(

            new Vector3(rndX, 20.0f, rndZ));

    }

    Destroy(gameObject, 1.5f);

}

```

멋진 폭발 효과를 주는 작은 함수가 여기 있습니다. 탱크의 Rigidbody 컴포넌트에 무작위 ExplosionForce 함수를 적용합니다. 모든 것이 올바르게 되면, 플레이어의 즐거움을 위해 탱크가 무작위 방향으로 공중으로 날아가는 것을 볼 수 있을 것입니다.

피해 입기 (Taking damage)

데모를 완성하기 위해 작은 세부 사항을 하나 더 추가해야 합니다. 총알에 맞았을 때 탱크가 피해를 입게 (take damage) 만들어야 합니다. 총알이 탱크의 충돌 영역에 들어올 때마다, Bullet 오브젝트의 damage 값에 따라 health 속성값이 감소합니다:

```

void OnCollisionEnter(Collision collision) {

```

```
// 체력 감소

if(collision.gameObject.tag == "Bullet") {

    health -=collision.gameObject.GetComponent

        <Bullet>().damage;

}

}
```

유니티에서 SimpleFSM.scene 파일을 열 수 있습니다. AI 탱크들이 순찰하고, 쫓고, 플레이어를 공격하는 것을 볼 수 있을 것입니다. 우리 플레이어의 탱크는 아직 AI 탱크로부터 데미지를 입지 않으므로 파괴되지 않습니다. 하지만 AI 탱크는 health 속성을 가지고 있으며 플레이어의 총알로부터 피해를 입기 때문에, 그들의 health 속성이 0에 도달하면 폭발하는 것을 볼 수 있습니다.

데모가 작동하지 않는다면, 인스펙터에서 SimpleFSM 컴포넌트의 값을 다양하게 변경해 보세요. 프로젝트의 규모에 따라 적절한 값이 달라질 수 있기 때문입니다:

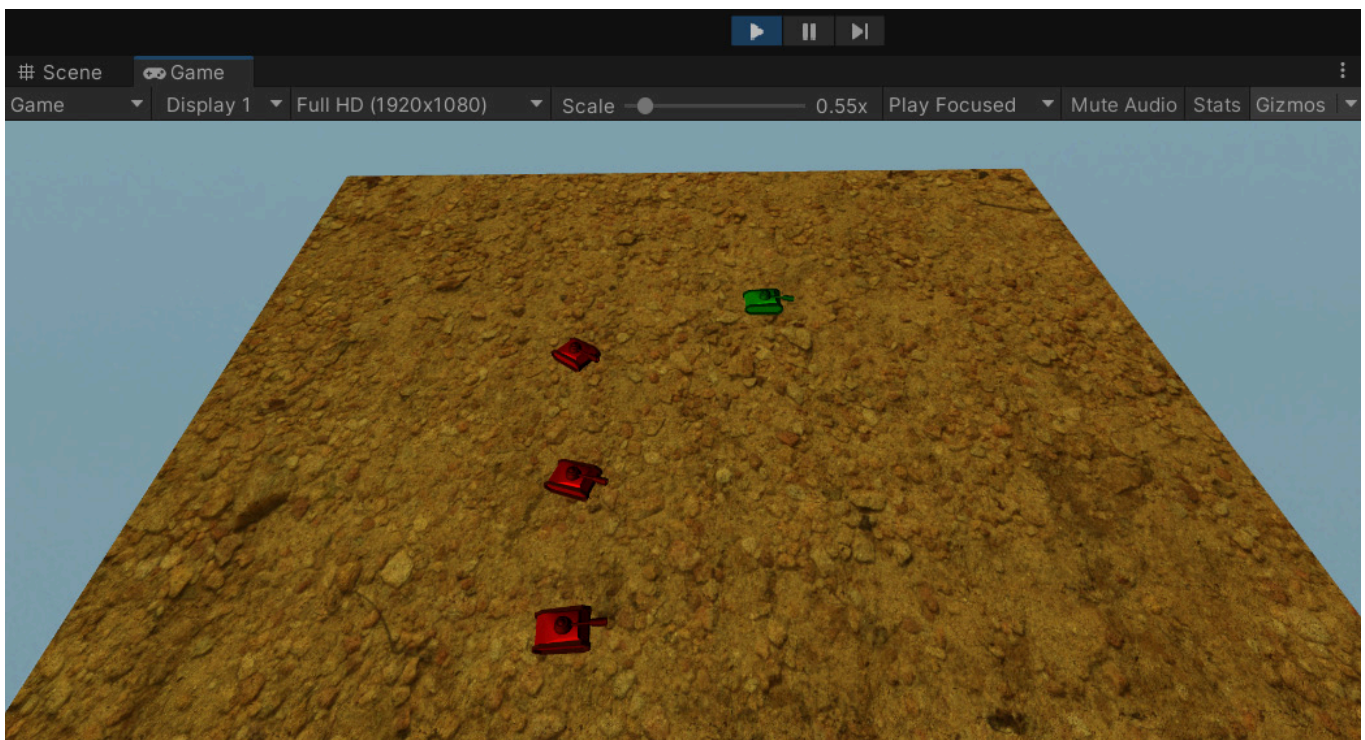


그림 2.10 – 작전 중인 AI 탱크

이 데모에서는 매우 간단한 FSM을 사용했지만, 이제 도전을 한 단계 높여 완전한 FSM 프레임워크를 구현할 차례입니다.

FSM 프레임워크 사용

여기서 사용할 FSM 프레임워크는 에릭 디센드(Eric Dybsend)의 Game Programming Gems 1의 3.1 장에 기초한 결정론적 유한 상태 기계(Deterministic Finite State Machine) 프레임워크를 조정한 것입니다. 여기서는 이 FSM과 이전에 만든 FSM의 차이점만 살펴볼 것입니다. 그렇기 때문에 도서 리포지토리의 Chapter02 폴더에 있는 예제 코드를 따라 하는 것이 중요합니다 (<https://github.com/PacktPublishing/Unity-Artificial-Intelligence-Programming-Fifth-Edition>). 특히, AdvancedFSM 씬(scene)을 살펴볼 것입니다.

이 섹션에서는 프레임워크가 어떻게 작동하는지, 그리고 이를 활용해 어떻게 탱크 AI를 구현할 수 있는지 연구할 것입니다. AdvancedFSM과 FSMState는 우리 프레임워크의 두 가지 주요 클래스입니다. 그럼 먼저 이들을 살펴보겠습니다.

AdvancedFSM 클래스

AdvancedFSM 클래스는 우리가 구현한 모든 FSMState 클래스를 관리하고, 트랜지션(전이)과 현재 상태를 지속적으로 업데이트합니다. 따라서 프레임워크를 사용하기 전에 가장 먼저 해야 할 일은 AI 탱크를 위해 구현할 전환(transitions)과 상태(states)를 선언하는 것입니다.

AdvancedFSM.cs 생성부터 시작하겠습니다:

```
using UnityEngine;

using System.Collections;

using System.Collections.Generic;

public enum Transition {

    None = 0, SawPlayer, ReachPlayer, LostPlayer, NoHealth,

}

public enum FSMStateID {

    None = 0, Patrolling, Chasing, Attacking, Dead,

}
```

여기서는 상태 집합(set of states)을 위한 열거형 enumerations과 전환 집합(set of transitions)을 위한 열거형 등 두 개의 열거형을 정의합니다. 그런 다음 FSMState 오브젝트들을 저장할 리스트 오브젝트를 추가하고, FSMState 클래스의 현재 ID와 현재 FSMState 자체를 저장할 두 개의 로컬 변수를 추가합니다.

AddFSMState 및 DeleteState 메서드는 각각 리스트에 FSMState 클래스의 인스턴스를 추가하고 삭제합니다. PerformTransition 메서드가 호출되면 전환에 따라 새로운 상태로 CurrentState 변수를 업

데이트합니다:

```
public class AdvancedFSM : FSM {

    private List<FSMState> fsmStates;

    private FSMStateID currentStateID;

    public FSMStateID CurrentStateID {

        get {

            return currentStateID;

        }

    }

    private FSMState currentState;

    public FSMState CurrentState {

        get {

            return currentState;

        }

    }

}
```

이제 클래스의 데이터 부분이 준비되었으므로 FSM 프레임워크의 내부 논리를 계속 진행할 수 있습니다.

FSMState 클래스

FSMState는 다른 상태로의 전환을 관리합니다. 이 클래스에는 전환과 상태의 키-값 쌍(key-value pairs)을 저장하는 map이라는 딕셔너리(dictionary) 오브젝트가 있습니다. 그래서 예를 들어, SawPlayer 전환은 Chasing 상태에 매핑되고, LostPlayer는 Patrolling 상태에 매핑되는 식입니다.

FSMState.cs 파일을 만들어 봅시다:

```
using UnityEngine;

using System.Collections;
```

```
using System.Collections.Generic;

public abstract class FSMState {

    protected Dictionary<Transition, FSMStateID> map =

        new Dictionary<Transition, FSMStateID>();

    // Continue...
```

AddTransition 및 DeleteTransition 메서드는 상태-전환 딕셔너리 map 오브젝트에서 전환을 추가하고 삭제합니다. GetOutputState 메서드는 map 오브젝트를 조회하여 입력 전환을 기반으로 상태를 반환합니다.

또한 FSMState 클래스는 자식 클래스들이 구현해야 하는 두 가지 추상 메서드를 선언합니다. 그들은 다음과 같습니다:

```
...

public abstract void CheckTransitionRules(Transform

    player, Transform npc);

public abstract void RunState(Transform player,

    Transform npc);

...
```

CheckTransitionRules 메서드는 해당 상태가 다른 상태로 전환을 수행해야 하는지 확인해야 합니다. 반면 RunState 메서드는 currentState 변수에 대한 작업(목적지를 향해 이동하거나 플레이어를 쫓거나 공격하는 등)의 실제 실행을 담당합니다. 두 메서드 모두 이 클래스를 사용하여 얻은 플레이어와 NPC(Non Playable Character) 엔티티의 트랜스폼 데이터를 필요로 합니다.

상태(State) 클래스들

이전 SimpleFSM 예제와 달리, 우리는 탱크 AI를 위한 상태들을 AttackState, ChaseState, DeadState, PatrolState와 같이 FSMState 클래스를 상속하는 별도의 클래스로 작성합니다. 이들 모두 CheckTransitionRules 및 RunState 메서드를 구현합니다. 예시로 PatrolState 클래스를 살펴보겠습니다.

PatrolState 클래스

이 클래스에는 생성자(constructor), CheckTransitionRules, RunState라는 세 가지 메서드가 있습니다. PatrolState.cs 파일에 PatrolState 클래스를 생성해 보겠습니다:

```
using UnityEngine;

using System.Collections;

public class PatrolState : FSMState {

    private Vector3 destPos;

    private Transform[] waypoints;

    private float curRotSpeed = 1.0f;

    private float curSpeed = 100.0f;

    private float playerNearRadius;

    private float patrolRadius;

    public PatrolState(Transform[] wp, float

        playerNearRadius, float patrolRadius) {

        waypoints = wp;

        stateID = FSMStateID.Patrolling;

        this.playerNearRadius = playerNearRadius;

        this.patrolRadius = patrolRadius;

    }

    public override void CheckTransitionRules(

        Transform player, Transform npc) {

        // Check the distance with player tank

        // When the distance is near, transition to chase

        // state

        if (Vector3.Distance(npc.position, player.position)
```

```

        <= playerNearRadius) {

            Debug.Log("Switch to Chase State");

            NPCTankController npcTankController =

                npc.GetComponent<NPCTankController>();

            if (npcTankController != null) {

                npcTankController.SetTransition(

                    Transition.SawPlayer);

            } else {

                Debug.LogError("NPCTankController not found

                                in NPC");

            }

        }

    }

    public override void RunState(Transform player,

        Transform npc) {

        // Find another random patrol point if the current

        // point is reached

        if (Vector3.Distance(npc.position, destPos) <=

            patrollRadius) {

            Debug.Log("Reached to the destination point\n

                        calculating the next point");

            FindNextPoint();

        }

```



```

        // Rotate to the target point

        Quaternion targetRotation =

            Quaternion.FromToRotation(Vector3.forward,

                destPos - npc.position);

        npc.rotation = Quaternion.Slerp(npc.rotation,

            targetRotation, Time.deltaTime * curRotSpeed);

        // Go Forward

        npc.Translate(Vector3.forward * Time.deltaTime *

            curSpeed);

    }

}

```

생성자(constructor) 메서드는 웨이포인트(waypoints) 배열을 받아 로컬 배열에 저장한 다음, 이동 및 회전 속도와 같은 속성(properties)을 초기화합니다. Reason 메서드(역주: 본문)의 CheckTransitionRules 메서드에 해당는 자신(AI 탱크)과 플레이어 탱크 사이의 거리를 확인합니다. 플레이어 탱크가 사거리 내에 있다면, 다음과 같이 작성된 NPCTankController 클래스의 SetTransition 메서드를 사용하여 전환(transition) ID를 SawPlayer 전환으로 설정합니다:

```

public void SetTransition(Transition t) {

    PerformTransition(t);

}

```

**위의 함수는 단지 AdvanceFSM 클래스의 PerformTransition 메서드를 호출하는 래퍼 메서드(wrapper method)일 뿐입니다. 그러면 해당 메서드는 Transition 오브젝트와 FSMState 클래스의 상태-전환(state-transition) 딕셔너리 map 오브젝트를 사용하여, 이 전환을 담당하는 상태로 CurrentState 변수를 업데이트합니다. Act 메서드(역주: 본문의 RunState 메서드에 해당)는 AI 탱크의 목적지 지점을 업데이트하고, 탱크를 해당 방향으로 회전시킨 후 앞으로 이동시킵니다.

다른 상태 클래스들 또한 각기 다른 추론(reasoning) 및 행동(acting) 절차를 가지며 이 템플릿을 따릅니다. 우리는 이전의 간단한 FSM 예제에서 이미 그것들을 살펴보았으므로, 여기서 다시 설명하지는 않겠습니다.

습니다. 이 클래스들을 스스로 어떻게 설정할 수 있을지 직접 파악해 보시기 바랍니다. 만약 막힌다면, 이 책과 함께 제공되는 에셋(assets)에 참고할 수 있는 코드가 포함되어 있습니다.

NPCTankController 클래스

탱크 AI의 경우, NPCTankController 클래스를 사용하여 NPC의 상태를 설정합니다. 이 클래스는 AdvanceFSM을 상속합니다:

```
private void ConstructFSM() {

    PatrolState patrol = new PatrolState(waypoints,

        playerNearRadius, patrollingRadius);

    patrol.AddTransition(Transition.SawPlayer,

        FSMStateID.Chasing);

    patrol.AddTransition(Transition.NoHealth,

        FSMStateID.Dead);

    ChaseState chase = new ChaseState(waypoints);

    chase.AddTransition(Transition.LostPlayer,

        FSMStateID.Patrolling);

    chase.AddTransition(Transition.ReachPlayer,

        FSMStateID.Attacking);

    chase.AddTransition(Transition.NoHealth,

        FSMStateID.Dead);

    AttackState attack = new AttackState(waypoints);

    attack.AddTransition(Transition.LostPlayer,

        FSMStateID.Patrolling);

    attack.AddTransition(Transition.SawPlayer,

        FSMStateID.Chasing);
```

```

        attack.AddTransition(Transition.NoHealth,

                                FSMStateID.Dead);

        DeadState dead = new DeadState();

        dead.AddTransition(Transition.NoHealth,

                                FSMStateID.Dead);

        AddFSMState(patrol);

        AddFSMState(chase);

        AddFSMState(attack);

        AddFSMState(dead);

    }

```

이것이 바로 우리 FSM 프레임워크 사용의 묘미(beauty)입니다. 상태들이 각자의 클래스 내에서 자체적으로 관리(self-managed)되기 때문에, NPCTankController 클래스는 현재 활성화된 상태의 Reason 및 Act 메서드만 호출하면 됩니다.

이러한 사실 덕분에 긴 if/else 및 switch 문장 목록을 작성할 필요가 없어집니다. 대신 우리의 상태들은 이제 그들 자체의 클래스에 깔끔하게 패키징(packaged)되며, 이는 대규모 프로젝트에서 상태와 상태 간의 전환 수가 점점 더 늘어남에 따라 코드를 더욱 관리하기 쉽게 만들어 줍니다:

```

protected override void FSMFixedUpdate() {

    CurrentState.Reason(playerTransform, transform);

    CurrentState.Act(playerTransform, transform);

}

```

이 프레임워크를 사용하는 주요 단계는 다음과 같이 요약할 수 있습니다:

1. AdvanceFSM 클래스에서 전환(transitions)과 상태(states)를 선언합니다.
2. FSMState 클래스를 상속받는 상태 클래스들을 작성한 다음, Reason 및 Act 메서드를 구현합니다.
3. AdvanceFSM을 상속받는 커스텀 NPC AI 클래스를 작성합니다.
4. 상태 클래스들로부터 상태(states)를 생성한 다음, FSMState 클래스의 AddTransition 메서드를 사용하여 전환 및 상태 쌍(pairs)을 추가합니다.

5. AddFSMState 메서드를 사용하여 AdvanceFSM 클래스의 상태 리스트(state list)에 해당 상태들을 추가합니다.
6. 게임 업데이트 주기(game update cycle) 내에서 CurrentState 변수의 Reason 및 Act 메서드를 호출합니다.

유니티에서 AdvancedFSM 씬을 자유롭게 테스트해 볼 수 있습니다. 이전의 SimpleFSM 예제와 동일한 방식으로 실행되지만, 코드는 이제 훨씬 더 체계적(organized)이고 관리하기 쉽게 되었습니다.

요약

이 장에서는 간단한 탱크 게임을 기반으로 Unity3D에서 상태 머신을 구현하는 방법을 배웠습니다. 먼저 **switch** 문을 사용하여 유한 상태 머신(FSM)을 구현하는 방법을 살펴보았습니다. 그런 다음 프레임 워크를 사용하여 AI 구현을 더 쉽게 관리하고 확장하는 방법을 공부했습니다.

다음 장에서는 무작위성(randomness)과 확률(probability)에 대해 살펴보고, 게임의 결과를 더 예측 불가능하게 만들기 위해 이들을 어떻게 활용할 수 있는지 알아보겠습니다.